Audio Classification: Yes/No Words

Mini Task 8 Report, Digital Speech Processing

Natiq Hassan Al Abboody | Student ID: 404131065
Instructor: Dr. Mohammad Khalooei | November 2026

1. Introduction

In this mini task we were asked to take an audio data set, look at its signals, sample rate and labels, and compare at least four classifiers, including an SVM and a neural network. I worked with the Yes/No corpus from the Kaldi project, which is also the data set used in the instructor's Kaldi tutorial [2], and I tried six classifiers.

My first results were close to 100% for every model, which left me with no real comparison to report. I therefore added two experiments: testing the models on words with added white noise, and training them on very few recordings. Most of the differences discussed in this report come from these two experiments.

2. The data set

The corpus [1] contains 60 mono recordings at 8 kHz with 16-bit samples, so the signal only covers 0–4 kHz. One male speaker says eight words in each file, and each word is "yes" or "no" in Hebrew (*ken* and *lo*). There is no separate transcript, because the labels are written in the file name: `0_1_1_0_0_1_1_0.wav` means no, yes, yes, no, no, yes, yes, no. Recordings last between about 5 and 7 seconds. OpenSLR and the instructor's tutorial say the language is Hebrew, while the README in the GitHub copy I downloaded says English. I followed OpenSLR. This makes no difference to the experiments, since the models only see the two word classes.

Figure 1 shows one recording. The words are easy to see in the waveform, with quiet gaps between them. The spectrogram also has faint horizontal lines at fixed frequencies between the words as well as during them. Speech would not behave like that, so they are probably a steady hum picked up during recording.

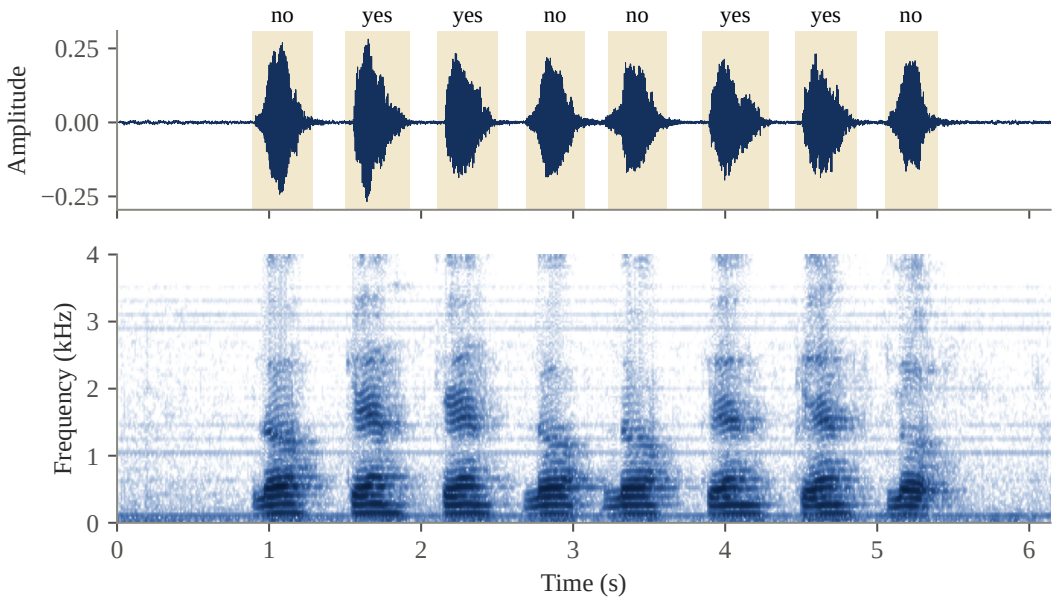


Figure 1: Waveform and spectrogram of `0_1_1_0_0_1_1_0.wav`. The shaded regions are the automatically detected words, with labels taken from the file name.

3. Method

3.1 Cutting recordings into words

Each file has eight labels, so it has to be cut into eight words before any word can be classified. I computed the short-time log energy, smoothed it, and tried many thresholds between the noise floor and the peak. Several thresholds usually give exactly eight regions. I took the middle one, because a threshold at the top of that range can cut off the weak end of a word. Each region was then extended by 50 ms on both sides.

This gave eight words in 59 files. The exception was `0_1_0_1_0_0_0_0.wav`. After the seventh word its energy stays at the noise floor until the end of the file, so it seems to contain only seven audible words although its name lists eight. I could not tell which label was missing, so I left this file out. The final set has 472 words from 59 recordings: 249 yes and 223 no. I did not check the segments by listening to all of them. The near-perfect results below do suggest that words and labels are correctly aligned, since a shifted label would appear as a classification error.

3.2 Features

Figure 2 compares one “yes” with one “no”. Their energy is distributed differently over frequency, which gives a first sign that spectral features should work well. I used MFCCs [3], which I wrote with NumPy and SciPy so that each step stayed visible: pre-emphasis with $\alpha = 0.97$, 25 ms Hamming frames with a 10 ms hop, a 256-point FFT, 26 mel filters and 13 cepstral coefficients. The delta coefficients are regression slopes over ± 2 frames:

$$d_t = \frac{\sum_{i=1}^{2} i\,(c_{t+i} - c_{t-i})}{2\sum_{i=1}^{2} i^2}. \tag{1}$$

The mel step is a filter bank, meaning a set of band-pass filters that splits a signal into sub-bands [6]. Windowed FFTs on overlapping frames already act as a bank of narrow filters, and the 26 triangular mel filters then group these FFT bins into wider bands. I only need this analysis side of a filter bank, because nothing has to be reconstructed. The DCT then turns the 26 log energies into cepstral coefficients.

Words have different lengths, but the classifiers need an input of fixed size. For each of the 39 coefficient tracks I therefore took the mean and standard deviation over the word, which gives 78 values. I removed the mean of c_0 in each word to reduce the effect of loudness, so that one of the 78 values is always zero. For comparison, I also trained an SVM on three simple time-domain values: duration, mean zero-crossing rate and RMS level.

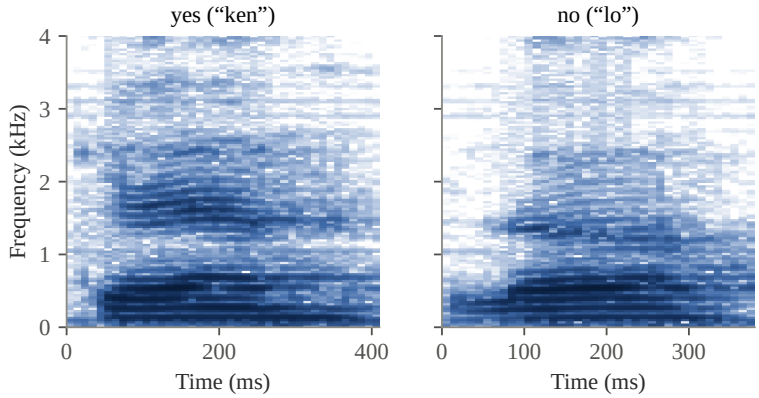


Figure 2: Spectrograms of one “yes” and one “no” from the recording in Figure 1.

3.3 Classifiers and evaluation

All models come from scikit-learn [5]. Features were standardised with values computed on the training data, except for the random forest, which does not need this. The tested settings were:

- **SVM** [4] with an RBF kernel: $C \in \{1, 10, 100\}$, $\gamma \in \{\text{scale}, 0.001\}$.
- **Neural network**: an MLP with hidden layers of (32) or (64, 32) units, and L_2 penalty $\alpha \in \{10^{-4}, 10^{-2}\}$.

- **Logistic regression**: $C \in \{0.01, 0.1, 1, 10\}$.
- **Random forest**: 200 trees, `max_features` set to sqrt or 0.3.
- **k-NN**: $k \in \{1, 3, 5, 9\}$.
- **Gaussian naive Bayes**: no settings to tune.

Words from one recording come from the same session, so I always kept a whole recording on one side of a split. Splitting by word would put very similar material in training and testing, and the scores would look better than they should.

I used two evaluations. The first is a hold-out split based on the tutorial, where the first 31 file names go to training and the rest to testing. After removing the excluded file, this gives 30 training and 29 test recordings. The second is stratified 5-fold cross-validation grouped by recording and repeated five times. In each training fold, a grouped 3-fold grid search chose the settings, so the test fold played no part in tuning.

For the noise test, I added white Gaussian noise to each word at 20, 10, 5, 0 and -5 dB SNR. Word boundaries came from the clean signal, so this test does not include segmentation errors. Each model was tested in two ways: trained on clean words only, and trained on clean words plus noisy copies at 20, 10, 5 and 0 dB (multi-condition training). I ran this test on the first repetition of cross-validation only. Finally, for the learning curve I kept 15 recordings for testing, trained on 1 to 20 of the others, and repeated this 20 times. Here each model used the settings chosen most often during cross-validation.

4. Results

4.1 Clean speech

Table 1 shows the results. Five models made no errors on the hold-out set, and all six are above 99.7% in cross-validation. The MFCC statistics clearly help, since the three time-domain values give only 81.9%, while always choosing the larger class gives 52.7%.

Table 1: Clean test results. CV columns: 5-fold cross-validation repeated 5 times, split by recording (25 folds, 2360 test predictions in all). Hold-out: 30 training and 29 test recordings (232 test words).

Classifier	Hold-out acc. (%)	Hold-out errors	CV acc. (%)	CV F1 (%)	CV errors
SVM (RBF)	100.00	0	99.79 ± 0.43	99.81	5
Neural network (MLP)	100.00	0	100.00 ± 0.00	100.00	0
Logistic regression	100.00	0	100.00 ± 0.00	100.00	0
Random forest	100.00	0	100.00 ± 0.00	100.00	0
k-NN	99.57	1	99.75 ± 0.45	99.77	6
Gaussian naive Bayes	100.00	0	99.79 ± 0.43	99.81	5
Majority class	–	–	52.75 ± 1.17	–	–
Time-domain features (SVM)	–	–	81.93 ± 4.78	–	–

Table 2 gives precision and recall with “yes” as the positive class, together with the confusion counts. F1 in Table 1 uses the same positive class. Nearly all errors are “no” words predicted as “yes”, which is why recall stays at 100% for most models while precision is slightly lower.

Table 2: Cross-validation precision and recall (%) with “yes” as the positive class, and the confusion counts summed over all 25 test folds.

Classifier	Precision	Recall	True no	False yes	False no	True yes
SVM (RBF)	99.60	100.00	1110	5	0	1245
Neural network (MLP)	100.00	100.00	1115	0	0	1245
Logistic regression	100.00	100.00	1115	0	0	1245
Random forest	100.00	100.00	1115	0	0	1245
k-NN	99.68	99.84	1111	4	2	1243
Gaussian naive Bayes	99.60	100.00	1110	5	0	1245

I would not rank the models from these numbers. All five errors of the SVM and all five of naive Bayes are the same word: the first word of `0_0_1_0_1_0_0_0.wav`, a "no" predicted as "yes" each time it falls in a test fold. k-NN misses this word four times, and misses the first word of `1_1_0_0_1_0_1_0.wav` twice. The first of these words has a normal length and level. It is still on the "no" side in Figure 3, but closer to the boundary than most "no" words, and logistic regression, the MLP and the random forest all get it right. So I think its label is correct and it is simply a harder example, although I did not find out exactly why.

A perfect hold-out score also proves less than it seems to. With no errors in 232 words, the "rule of three" only suggests an error rate below about 1.3%, and even that assumes that the words are independent, which is not fully true here. This split has another weakness that I noticed afterwards. Sorted file names put every file starting with 0 in training and every file starting with 1 in testing, so the share of "yes" words differs (46% in training, 59% in testing). Cross-validation is the more reliable of the two evaluations for this reason.

Figure 3 helps to explain why the task is so easy. The words form two groups, and the sign of the first principal component alone separates all 472 words, even though PCA never sees the labels. This component holds only 22% of the total variance, so the difference between the two words is a strong, simple direction in the feature space, and almost any classifier can find it.

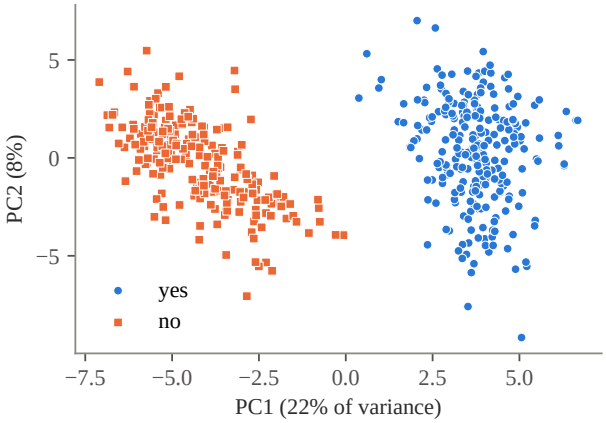


Figure 3: All 472 words on the first two principal components of the standardised features.

4.2 Noise

Figure 4 and Table 3 show the noise results. With clean training, the models are still good at 20 dB but drop quickly after that. Naive Bayes is the best at 10 dB and then falls sharply. From 5 dB downwards logistic regression stays ahead, with 69.1% at 0 dB. At the same level the SVM gives 54.7%, the MLP 54.1%, and the other models are also close to the 52.7% of the majority class. I expected the more flexible models to do better here, so this result surprised me. One possible reason is that a linear model depends less on the fine details of clean data, but I did not test this.

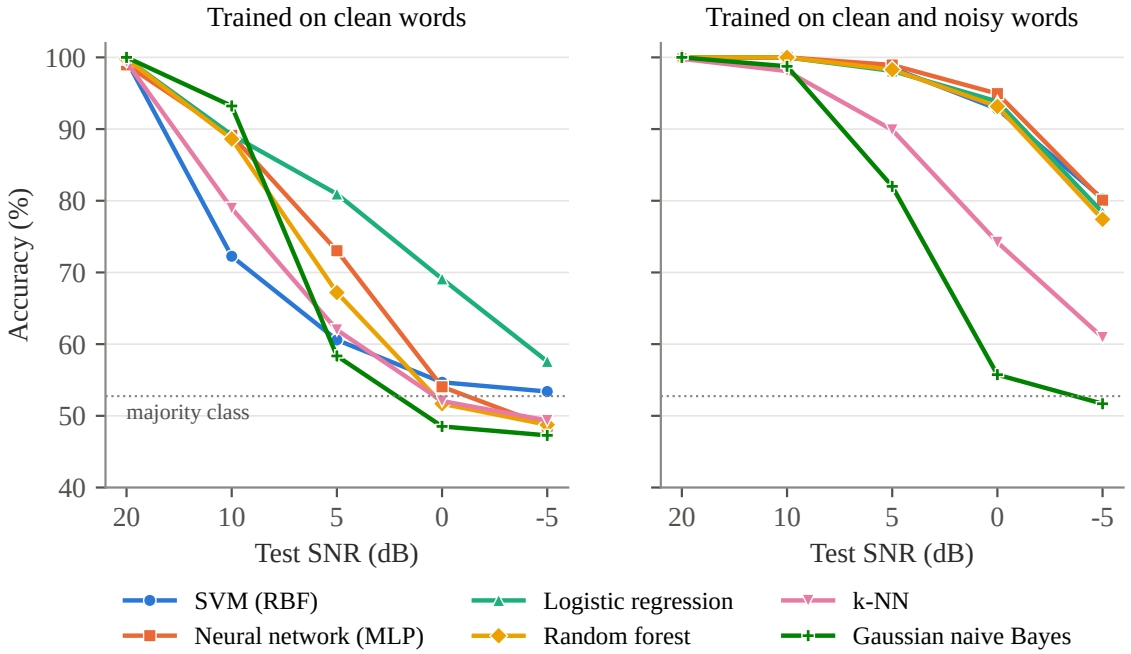


Figure 4: Accuracy against test SNR. Both panels use the same vertical scale, and each point is the mean of 5 folds.

Multi-condition training changes the picture far more than the choice of model does. At 0 dB the MLP reaches 94.9% and the SVM 92.8%, with logistic regression and the random forest in the same range. The standard deviation across folds is 1–3 points for these four, so I cannot say that one of them is better. k-NN and naive Bayes gain much less, with 74.2% and 55.7% at 0 dB. Note that the noise levels from 20 to 0 dB and the noise type were all seen in training, so this is a favourable case. The only unseen level is −5 dB, where the four better models give about 77–80% (SVM 80.3%, MLP 80.1%).

Table 3: Accuracy (%) on noisy test words, mean ± std over 5 folds.

	Trained on clean words			Trained on clean and noisy words		
Classifier	10 dB	5 dB	0 dB	10 dB	5 dB	0 dB
SVM (RBF)	72.3 ± 9.4	60.6 ± 7.3	54.7 ± 1.5	100.0 ± 0.0	98.5 ± 1.2	92.8 ± 2.5
Neural network (MLP)	89.1 ± 3.1	73.0 ± 7.9	54.1 ± 5.5	100.0 ± 0.0	98.9 ± 0.7	94.9 ± 1.3
Logistic regression	89.2 ± 2.5	80.9 ± 4.1	69.1 ± 6.7	100.0 ± 0.0	98.1 ± 1.1	93.8 ± 2.3
Random forest	88.6 ± 7.3	67.2 ± 10.0	51.7 ± 4.5	100.0 ± 0.0	98.3 ± 1.2	93.2 ± 2.2
k-NN	78.9 ± 8.8	62.0 ± 10.0	52.1 ± 7.1	98.1 ± 1.6	89.9 ± 3.3	74.2 ± 3.9
Gaussian naive Bayes	93.2 ± 7.3	58.4 ± 19.3	48.5 ± 3.3	98.8 ± 1.1	82.0 ± 4.6	55.7 ± 3.7

4.3 Amount of training data

Figure 5 shows how little data this task needs. With one recording, which means eight words, k-NN reaches 96.3% and the MLP 94.8%. All models pass 99% with five recordings. The SVM starts lowest, with 62.7% on one recording, 90.9% on three and 99.7% on five. Its settings ($C = 1$, $\gamma = 0.001$) were mostly chosen on about 47 recordings, and they are probably too strongly regularised for a few words, so I do not read this curve as a general weakness of the SVM. Since these settings were chosen on recordings that can also appear in this test, the curve is a rough guide and not a strict evaluation.

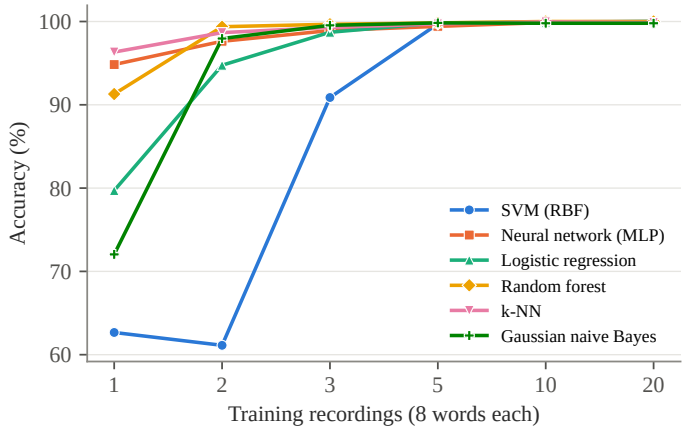


Figure 5: Accuracy on 15 held-out recordings against training size, averaged over 20 random draws.

4.4 Computation time

Table 4 shows that time is not a problem for any model. The slowest at prediction, the random forest, takes about 0.1 ms per word, and fitting takes less than two seconds even with the grid search. These times come from one machine and will change on another one.

Table 4: Average cost per training fold and per word on the 2-core machine used here. Fit time includes the grid search, and naive Bayes has no grid, so its figure is not directly comparable.

Classifier	Fit with tuning (s)	Prediction (ms/word)
SVM (RBF)	0.10	0.014
Neural network (MLP)	0.89	0.007
Logistic regression	0.06	0.006
Random forest	1.71	0.103
k-NN	0.05	0.012
Gaussian naive Bayes	0.00	0.006

4.5 Filter bank energies against MFCCs

Since MFCCs are only a DCT of the log filter bank energies, I wanted to see whether the DCT step helps here. I repeated the experiments with the 26 log energies, using the same statistics and deltas (156 values), the same splits and the same noise. In each word I subtracted one overall mean as a level correction. Table 5 compares the two features.

Table 5: Accuracy (%) with 26 log filter bank energies (FB) and with MFCCs, on the same splits and the same noise.

	Clean words		10 dB, clean training		0 dB, multi-condition	
Classifier	FB	MFCC	FB	MFCC	FB	MFCC
SVM (RBF)	99.66	99.79	52.7	72.3	90.5	92.8
Neural network (MLP)	99.62	100.00	52.7	89.1	90.3	94.9
Logistic regression	100.00	100.00	52.7	89.2	88.4	93.8
Random forest	100.00	100.00	67.8	88.6	91.4	93.2
k-NN	99.66	99.75	60.4	78.9	62.6	74.2
Gaussian naive Bayes	99.79	99.79	74.5	93.2	47.9	55.7

On clean words there is again almost no difference. With clean training, MFCCs are clearly better at 10 dB for all six models. At 0 dB the random forest and naive Bayes actually score higher with filter bank energies (59% and 58%), but both are still poor. With filter bank energies, the SVM, the MLP and logistic regression all give exactly the majority rate of 52.7% at 10, 5 and 0 dB. I checked one fold and found that they label every noisy

word as “yes”, so even mild noise moves the whole set across their decision boundary. My guess is that white noise lifts the weak bands together, and the correlated energies carry this shift straight into the model, but I did not test this. Multi-condition training removes most of the problem, although MFCCs stay about 2–5 points ahead at 0 dB for the four better models, and part of this gap is within the spread across folds.

5. Discussion

On clean words I cannot say that one classifier is better, because all of them almost reach the ceiling. The noise experiment gave me two more useful results. Training on noisy data helped much more than changing the classifier, and the most flexible models were not the most robust when they had only seen clean words. If I had to choose one model for this data, I would use logistic regression or the MLP with multi-condition training. Their results cannot be separated here, so I would prefer logistic regression because it is simpler.

Both of my feature sets depend on a hand-designed filter bank. wav2vec 2.0 [7] takes a different route and learns its features from the raw waveform with a CNN encoder. During pre-training it hides about 49% of the encoded time steps, which is similar to masked language modelling in BERT, where hidden words are predicted from their context [8]. A Transformer then has to pick the true quantised vector for each masked step from a set that also holds 100 distractors. After this, little labelled data is needed: with ten minutes of labelled speech the paper reports a WER of 4.8/8.2 on the clean/other test sets. My learning curve shows that labels are not the difficulty on this corpus, so the gain from such a model would show on a harder task with many speakers. I did not try it here. It would need a pre-trained model and 16 kHz audio, while this corpus is at 8 kHz.

These results have clear limits:

- All recordings come from one speaker, so nothing here tells us how the models would work on another voice.
- The task has only two short words, which is very far from real speech recognition.
- The noise is white noise added after segmentation. In a real noisy recording the energy-based segmentation would probably fail before the classifier does, and this is one reason a proper VAD matters.
- Each SNR uses a single noise realisation and only 5 folds, so small differences mean very little.
- I used a small MLP on summary features and did not try a CNN on spectrograms, since the clean task leaves little room for improvement.

6. Conclusion

I segmented the Yes/No recordings into words, described each word with MFCC statistics, and compared six classifiers using splits made per recording. All six reached at least 99.7% on clean words, compared with 81.9% for the simple time-domain values. Noise gave a better comparison: at 0 dB most models trained on clean words were close to chance, while multi-condition training brought the SVM, MLP, logistic regression and random forest above 92%. Removing the DCT did not matter on clean words, but MFCCs were more stable under noise. The main thing I learned is that a very high score on an easy data set says little about a model, and that how we split the data and what we train on can matter more than the classifier. The next step would be a data set with many speakers, such as Google Speech Commands, using pre-trained wav2vec 2.0 features as well as MFCCs.

References

[1] “Yes/No,” Open Speech and Language Resources, SLR1. `https://www.openslr.org/1/`

[2] M. Khalooei, “Kaldi Speech Recognition Toolkit Tutorial.” `https://github.com/khalooei/kaldi-tutorial`

[3] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Trans. Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] "Filter bank," Wikipedia. https://en.wikipedia.org/wiki/Filter_bank

[7] A. Baevski, H. Zhou, A. Mohamed and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," arXiv:2006.11477, 2020.

[8] "Masked language model," Coursera Articles. https://www.coursera.org/articles/masked-language-model